Principles of Machine Learning

Mariette Schönfeld (session 1,3,5) *mariette.schoenfeld@kuleuven.be*
Andrea Mirenda session (2,4) *andrea.mirenda@kuleuven.be*
Isidoro Tamassia (session 6,7) *isidoro.tamassia@kuleuven.be*

Fall, 2026

Exercise Session 1: Decision Trees and k-NN

1.1 Logical Formulas as Decision Trees

Let the left subtree be the *true branch*. Draw decision trees for the following formulas.

a. $A \wedge \neg B$

b. $\neg A \vee B$

c. $A \vee (B \wedge C)$

d. $(\neg A \wedge B) \vee (A \wedge \neg B)$

e. $(A \vee B) \wedge (C \vee D \vee \neg E)$

f. $(A \vee B \vee C) \wedge (D \vee E \vee F)$

Solution 1.1 See Figure 1. ◇

1.2 The Decision Surface of a Tree

Consider a data set with two *numeric* attributes a_1 and a_2 and one nominal target attribute c with two possible values: $\oplus$ and $\ominus$. The training examples are shown in Figure 2.

a. Find a decision tree that classifies all training examples correctly.

b. Draw the decision surface of this tree in Figure 2.

Solution 1.2

a. The decision tree is shown in Figure 3.a.

b. The decision surface is shown in Figure 3.b.

◇

1.3 Measuring Test Quality: Entropy, Information Gain, and Variance Reduction

Consider the following table of training examples:

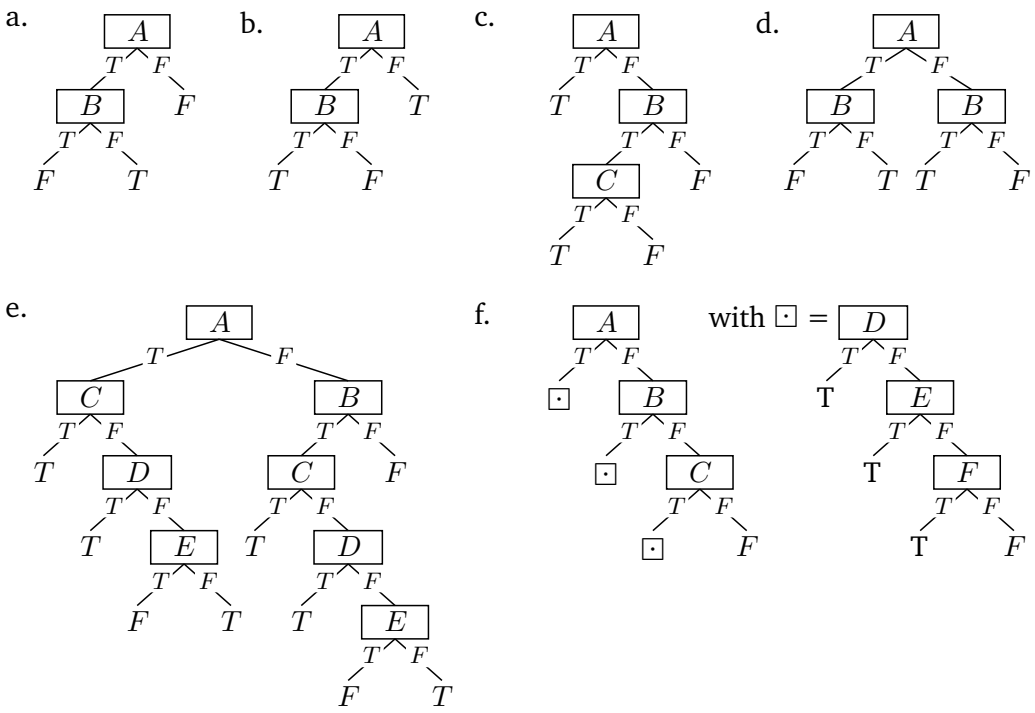


Figure 1: Solution for exercise 1.1.

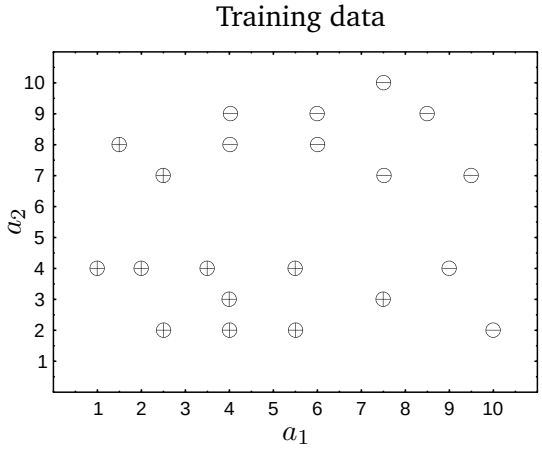


Figure 2: Training data for question 1.2.

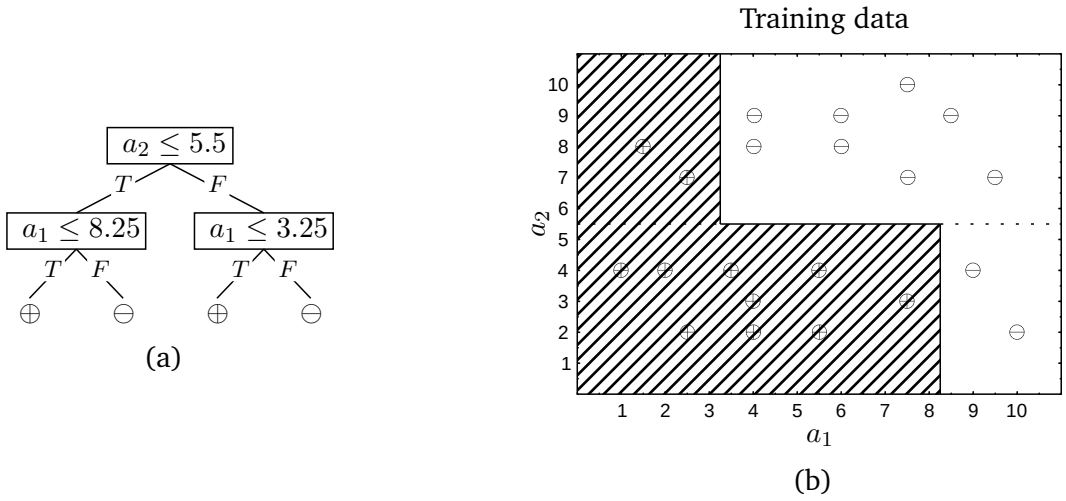


Figure 3: Decision tree and surface for exercise 1.2

Instance	a_1	a_2	y_1	y_2
1	T	T	$\oplus$	1.0
2	T	T	$\oplus$	1.0
3	T	F	$\ominus$	1.0
4	F	F	$\oplus$	5.0
5	F	T	$\ominus$	6.0
6	F	T	$\ominus$	5.5

a. What is the entropy with respect to the y_1 target attribute?

b. What is the information gain of splitting on attribute a_2 (easy) and a_1 (use calculator) when y_1 is the target attribute? Which attribute is the best choice and why?

c. What is the variance reduction of splitting on a_1 and a_2 when y_2 is the target attribute?

d. When would you use information gain and when would you use variance reduction?

Solution 1.3 Let data $D = \{\boldsymbol{x}_i, y\}_{i=1}^n$, with $y \in \{C_1, \ldots, C_K\}$ and D_k, $k = 1, \ldots, K$, the instances belonging to class k. Then class entropy is defined as:

$$\mathrm{CE}(D) = -\sum_{k=1}^{K} \frac{|D_k|}{|D|} \log_2(\frac{|D_k|}{|D|})$$

Information gain is defined as, with t a test with O outcomes (i.e., branches), and D_o the instances that belong to outcome o:

$$\mathrm{IG}(D, t) = \mathrm{CE}(D) - \sum_{o=1}^{O} \frac{|D_o|}{|D|} \mathrm{CE}(D_o).$$

Variance reduction is defined as:

$$\mathrm{VR}(D, t) = \mathrm{Var}(D) - \sum_{o=1}^{O} \frac{|D_o|}{|D|} \mathrm{Var}(D_o).$$

a. Equal probabilities $\rightarrow$ entropy = 1.

b. For a_2, entropy stays 1 $\rightarrow$ gain = 0. For a_1:
$\mathrm{CE}(D_{a_1=T}) = -\frac{2}{3}\log_2(\frac{2}{3}) - \frac{1}{3}\log_2(\frac{1}{3}) = 0.5283208 + 0.389975 = 0.9182958$ $(= \mathrm{CE}(D_{a_1=F}))$
$\mathrm{IG}(D, a_1) = 1 - \frac{1}{2}0.9182958 - \frac{1}{2}0.9182958 = 0.0817042$

A split on a_1 is preferred. The *true* branch contains 2/3 positive instances, and the *false* branch contains 2/3 negative instances. There is a majority class in each branch.

c. In this course we typically use the *unbiased sample variance* as our definition for variance:

$$\frac{1}{N-1}\sum_{i=1}^{N}(x_i-\bar{x})^2$$

Variance reduction for target y_2:

- $\mathrm{Var}(D) = 6.17$
- $\mathrm{Var}(D[a_1 = T]) = 0$
- $\mathrm{Var}(D[a_1 = F]) = 0.25$
- $\mathrm{Var}(D[a_2 = T]) = 7.56$
- $\mathrm{Var}(D[a_2 = F]) = 8$
- $\mathrm{VR}(D, a_1) = 6.05 \leftarrow a_1$ has the best variance reduction
- $\mathrm{VR}(D, a_2) = -1.53$

The unbiased sample variance is different from the *population variance*, which is calculated as follows:

$$\frac{1}{N}\sum_{i=1}^{N}(x_i-\mu)^2$$

and will give:

- $\mathrm{Var}(D) = 5.15$
- $\mathrm{Var}(D[a_1 = T]) = 0$
- $\mathrm{Var}(D[a_1 = F]) = 0.167$
- $\mathrm{Var}(D[a_2 = T]) = 5.67$
- $\mathrm{Var}(D[a_2 = F]) = 4$
- $\mathrm{VR}(D, a_1) = 5.06 \leftarrow a_1$ has the best variance reduction
- $\mathrm{VR}(D, a_2) = 0.0312$

In previous courses you may have learned the population variance as the 'true' definition. The unbiased sample variance tends to give a better estimate for the true variance when your sample size is small, as the difference between $\frac{1}{N-1}$ and $\frac{1}{N}$ diminishes for large values of N. Neither is incorrect here though.

◇

1.4 Constructing a Tree: ID3

The ID3 algorithm is a name for the *top-down induction of decision tree (TDIDT)* algorithm that uses *information gain* as a heuristic.

Instance	**Sky**	**AirTemp**	**Humidity**	**Wind**	**Water**	**Forecast**	**EnjoySport**
1	sunny	warm	normal	strong	warm	same	yes
2	sunny	warm	high	strong	warm	same	yes
3	rainy	cold	high	strong	warm	change	no
4	sunny	warm	high	strong	cool	change	yes
5	sunny	warm	normal	weak	warm	same	no

a. Construct a decision tree that correctly classifies the **EnjoySport** attribute using only instances 1-4 from the table above. Specifically, think of ways in which the positive instances can be separated from the negative instances by formulating split conditions on the other attributes.

b. We will now simulate the ID3 algorithm using all 5 instances.

Compute the class entropy for the entire dataset S:

	$+$	$-$	Entropy
S	3	2	

Compute the split heuristic for each attribute and select the attribute for the root node:

Attribute	**Values**	$\oplus$	$\ominus$	**Entropy**	**IG**
Sky	sunny	3	1	0.811	0.32
	rainy	0	1	0.0	
AirTemp	warm				
	cold				
Humidity	normal	1	1	1.0	0.02
	high	2	1	0.918	
Wind	strong				
	weak				
Water	warm				
	cool				
Forecast	same				
	change				

Manually execute the ID3 algorithm until you obtain a complete decision tree.

Solution 1.4

a. Sky: sunny $\rightarrow$ yes; rainy $\rightarrow$ no.
Alternative: AirTemp: warm $\rightarrow$ yes; cold $\rightarrow$ no

b. $S = [3\oplus, 2\ominus]$, entropy is 0.97.

Attribute	Values	$\oplus$	$\ominus$	Entropy	IG
Sky	sunny	3	1	0.811	0.32
	rainy	0	1	0.0	
AirTemp	warm	3	1	0.811	0.32
	cold	0	1	0.0	
Humidity	normal	1	1	1.0	0.02
	high	2	1	0.918	
Wind	strong	3	1	0.811	0.32
	weak	0	1	0.0	
Water	warm	2	2	1.0	0.17
	cool	1	0	0.0	
Forecast	same	2	1	0.918	0.02
	change	1	1	1.0	

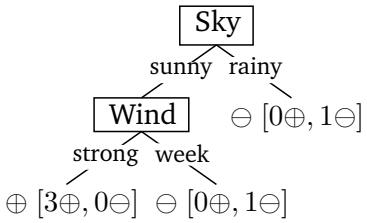
Three attributes have equal information gain: *Sky*, *AirTemp*, and *Wind*. Any of them can be selected for the root node of the decision tree.

Assume Sky is selected. The right node corresponding to *Sky* = *rainy* is pure, i.e. it contains examples of one class only ($-$). It becomes a leaf node.

Computing the information gain for the left node (*Sky* = *sunny*) yields the following values:

Attribute	IG
AirTemp	0.00
Humidity	0.31
Wind	0.81
Water	0.12
Forecast	0.12

Wind has the highest IG. Splitting on *Wind* results in two pure leaves, and ID3 terminates.



◇

1.5 Predicting Probabilities with Decision Trees

Formulate a strategy that uses a decision tree to define a conditional probability distribution of the discrete target attribute. For example, for the dataset in the previous exercise, the algorithm should not just predict a class (i.e., *yes* or *no*), but rather it should predict a probability value (e.g. 80% *yes*, and 100%-80%=20% *no*).

What would be the advantages and disadvantages of this approach?

Solution 1.5 You can use the instance counts in the leafs of the tree to construct a probability distribution in each leaf. While this is easy to implement, this approach tends to *overfit* its probability

estimates, with in the extreme case (i.e. perfect classification) having only 0 and 1 probabilities.

An effective fix is to use the Laplace correction. More information can be found in this paper:

> Provost, F., Domingos, P. Tree Induction for Probability-Based Ranking. Machine Learning 52, 199–215.

◇

1.6 Instance-Based Learning: k-NN

Instance based learning consists of storing examples and comparing new instances with the examples, taking the target values of the examples nearest to the new instance to compute a prediction for the new instance.

Instance based classification requires a distance measure. When data can be represented as n-dimensional vectors with numeric or symbolic components, a generally usable distance function is

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_j w_j \cdot \delta(x[j], y[j])^2},$$

where $x[j]$ is the jth attribute value of instance x. For symbolic values, δ is defined by $\delta(x, y) = 0$ if $x = y, 1$ otherwise; and for numerical values, $\delta(x, y) = |x - y|$.

Note that for purely numerical vectors, with $w_i = 1$ we obtain the Euclidean distance, whereas for purely symbolic vectors the same weights give us the square root of the Hamming distance (square root is monotonically increasing, so relative ordering is unaffected).

In this exercise the instance space is $\{\text{true}, \text{false}\} \times \mathbb{R}$. Unless specified otherwise, use the function mentioned above as the distance function.

Consider the following points.

Instance	**A**	**B**	**Class**
1	true	1	$\oplus$
2	false	3	$\oplus$
3	true	5	$\oplus$
4	false	2	$\ominus$

Use $w_j = 1$ for the first three tasks:

a. Classify (false, 4) with nearest-neighbors or 1-NN.

b. Classify (false, 4) with 3-NN.

c. Classify (false, 4) using the prototypes $\boldsymbol{p}^{\oplus}$ = (true, 3, $\oplus$) and $\boldsymbol{p}^{\ominus}$ = (false, 2, $\ominus$).

d. Repeat c. but *min-max*-normalize attribute **B** and use the following weights for each attribute:

$$w_j = 1 - \frac{1}{n} \sum_{i \in \{\oplus, \ominus\}} \sum_{x : x^y = i} \delta(\boldsymbol{p}_j^i, x_j),$$

with $\boldsymbol{p}_j$ the jth component value of prototype $\boldsymbol{p}$ and x^y the class of instance x. What is the interpretation of these weights?

Solution 1.6

a. $d(x_1, (\text{false}, 4)) = \sqrt{1 + 9} = 3.16$
$d(x_2, (\text{false}, 4)) = \sqrt{1} = 1$
$d(x_3, (\text{false}, 4)) = \sqrt{1 + 1} = 1.41$
$d(x_4, (\text{false}, 4)) = \sqrt{4} = 2$
closest to $x_2 \rightarrow$ positive class

b. closest to x_2, x_3 and $x_4 \rightarrow 2\oplus, 1\ominus \rightarrow$ positive class

c. $d(\boldsymbol{p}^{\oplus}, (\text{false}, 4)) = \sqrt{1+1} = 1.41$
$d(\boldsymbol{p}^{\ominus}, (\text{false}, 4)) = \sqrt{0+4} = 2$
$\rightarrow$ positive class

d. After rescaling: $(\text{false}, 4) \rightarrow (\text{false}, 0.75)$, $\boldsymbol{p}^{\oplus} = (\text{true}, 0.5)$, $\boldsymbol{p}^{\ominus} = (\text{false}, 0.25)$.
$w_{\mathbf{A}} = 1 - 0.25 \cdot [\underbrace{(0+1+0)}_{\boldsymbol{p}^{\oplus}} + \underbrace{(0)}_{\boldsymbol{p}^{\ominus}}] = 0.75$
$w_{\mathbf{B}} = 1 - 0.25 \cdot [\underbrace{(0.5+0+0.5)}_{\boldsymbol{p}^{\oplus}} + \underbrace{(0)}_{\boldsymbol{p}^{\ominus}}] = 0.75$
$d(\boldsymbol{p}^{\oplus}, (\text{false}, 0.75)) = \sqrt{0.75 \times 1^2 + 0.75 \times 0.25^2} = 0.893$
$d(\boldsymbol{p}^{\ominus}, (\text{false}, 0.75)) = \sqrt{0.75 \times 0 + 0.75 \times 0.5^2} = 0.433$

$\rightarrow$ negative class

Interpretation: assign larger weights to attributes, given one prototype (equals class in this case), that have mostly similar values; assign smaller weights to attributes that vary greatly within one prototype.

$\diamond$

1.7 Visualizing 1-NN: Voronoi maps

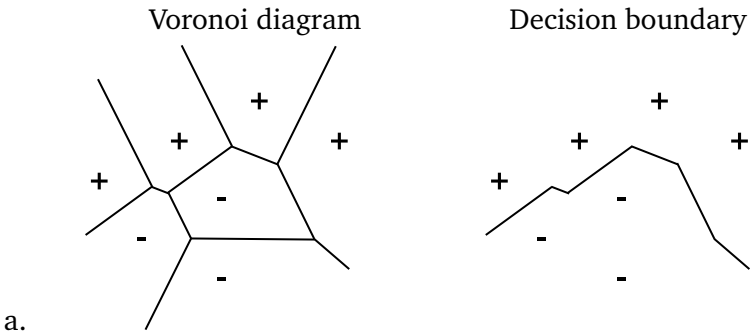
Sketch a Voronoi map for the 1-NN method for the following examples, and indicate the decision surface that results.

What would the decision surface look like if you used the prototype based approach (replace the set of positive examples by a single example lying in the middle of them, same for the negatives)?

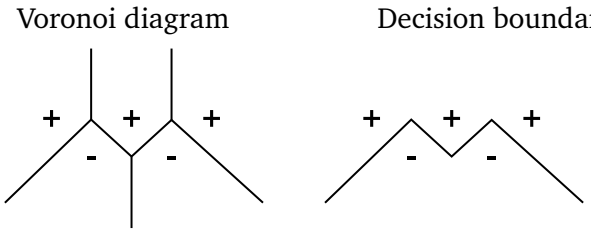
Think about some other differences between the prototype based approach and storing each individual example.



Solution 1.7 With prototypes: faster when making predictions (fewer examples to compare with); stronger and more explicit generalisations are made; flatter decision surfaces are obtained.



b.



◇

1.8 Curse of Dimensionality

a. The k-NN algorithm's performance deteriorates as the number of attributes increases. Why is this true? What are possible solutions to this problem? Are decision trees affected in a similar way?

b. With a lot of attributes, spurious correlations can become a problem. How do k-NN and decision trees (mis)-use spurious correlations? What information could you use to determine whether a decision tree is affected by spurious correlations?

Solution 1.8

a. With respect to prediction time, finding the k nearest neighbors becomes expensive. In terms of prediction performance: with very high dimensions, all instances are basically equidistant, i.e., distances become more difficult to interpret. In addition, irrelevant features might cause two instances to be far apart while the values of the predictive features are close. Detecting this is hard.

   Possible solutions are:

   - Speeding up nearest neighbors search $\rightarrow$ clever (approximate) data structures: KD-trees, locality sensitive hashing (see Big Data Analytics Course)
   - Reducing the number of attributes: feature selection methods, compression using dimensionality reduction techniques (covered later in the course).

   Decision trees are not affected in the same way. Finding a split test will take more time as more potential split candidates need to be evaluated. In contrast to k-NN, irrelevant features won't affect the predictions as much as they are unlikely to be chosen as a split attribute (unless there are spurious correlations).

b. As the number of attributes grows, it becomes more likely that some random correlations occur between input attributes and the target attribute in the training set that aren't present in the actual data distribution. A decision tree could *wrongly* split on these attributes, which would negatively affect the performance on the test set because overfitting occurs. For k-NN, spurious correlations could affect the distance function.

◇

1.9 Programming Exercise: Experimenting with the Effects of Normalization and Irrelevant Features on k-NN

You can find a Python script `mnist_knn.py` and a dataset `mnist.h5` on Toledo. Download both files and store them in the same folder. Make sure you have the following Python packages installed:

- `numpy`
- `pandas`
- `tables` (support for the HDF5 data format)
- `scikit-learn`

- `matplotlib`

You can use `pip` or `Conda` for installing Python packages. On Linux, you likely can install these packages using your package manager.

Answer the questions in the Python file.

Solution 1.9

- Do the labels match?

  Yes, in most cases the nearest examples have matching labels. You can use the majority label for the prediction. Of course, varying the number k will affect the performance.

- What is the effect of normalization

  Normalization ensures that each feature has a *fair* contribution to the distance. If the dataset used in this exercise had some pixels values expressed in *'pixels'* with values between 0-255, and other pixel values expressed in *'milli-pixel'* with values between 0 and 1, then clearly the *pixels* would contribute more to the distance than the *milli-pixels*.

  The performance in the example goes down due to this effect, though **no information is lost**, the un-normalization step can simply be reversed.

- What is the effect of irrelevant features?

  The irrelevant features also affect the distance function. Due to the curse of dimensionality, the more features, the more difficult distances are to interpret.

- The biggest downsides of k-NN is:
  - We need to store the full training data set.
  - The construction of the index (e.g. a KD-tree in the example) is quite expensive.
  - Even though we have an index, finding the nearest neighbors of an example is still quite slow compared to inference in a decision tree.

◇